

Course: Analysis of Algorithms (self-paced)

Built around **Sedgewick & Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. (2013)** — the Princeton "AofA" text. This is *not* a Big-O crash course; it's about computing the **precise, average-case cost** of algorithms. It picks up exactly where a data-structures / Big-O class stops and asks: "OK, but what is the *actual* expected number of operations, to leading term and constant?"

Stance — self-contained and thorough. This course is taught from the ground up, assuming *no* prior algorithms course: every prerequisite (Big-O basics, combinatorics, probability, recurrences) is built here, not referenced away. If a part is redundant with what you already know, say "**skip**" and we jump ahead — but the default is to teach it properly.

How this course works

- **Foundations first, then one module per chapter.** Module 0 builds the math toolkit the book assumes; then Module 1 = Ch 1 ... Module 9 = Ch 9, following the book's progression.
- Each module ships as: **lecture notes** (`notes/` , my own plain-text distillation — no LaTeX) + a **problem set with worked solutions** (`problem-sets/`). The book's own end-of-section exercises are extracted per chapter under `exercises/` for reference and self-testing.
- **Expansions ("sub-chapters") open on demand.** When a topic bites — combinatorics, a gnarly recurrence, a dense chapter — we spin off a focused expansion file (e.g. `notes/module-03a-master-theorem.md`) instead of overloading the main module.
- Progress is tracked in `PROGRESS.md` . We move at your pace, module by module, like the Infi 2 flow.

Calibration notes (for me)

- Background: calculus (Infi 1-2) solid; **combinatorics shallow and rusty** → we open with a full **Module 0 (Foundations)** building sums, counting, binomial coefficients, and discrete probability from scratch, and keep re-teaching combinatorial steps inline thereafter.
- No deadline. Goal is genuine depth in algorithmic analysis, motivated by a data-structures course.

The roadmap

Part 0 — Foundations (self-contained prerequisites)

Module	Topic	Why it matters
0	Foundations toolbox — sums & Σ -manipulation, counting, binomial coefficients + identities, discrete probability & expectation, harmonic numbers	Everything the book leans on but doesn't teach. Built from scratch so the rest stands on solid ground. Skip any part you already own.

Part I — The mathematical toolkit (Ch 1-4)

Module	Chapter	Topic	Why it matters
1	1	Analysis of Algorithms	The whole philosophy + the quicksort analysis as the running example. Reframes Big-O into <i>exact expected cost</i> .
2	2	Recurrence Relations	Recursive algorithms → recurrences. First-order, linear, divide-and-conquer, the Master theorem done properly.
3	3	Generating Functions	The central new tool: encode a whole sequence as one function, solve recurrences mechanically.
4	4	Asymptotic Approximations	Makes O-notation rigorous; how to turn exact-but-ugly answers into clean $\sim$ estimates (Euler-Maclaurin, Laplace).

Part II — The pivot (Ch 5)

Module	Chapter	Topic	Why it matters
5	5	Analytic Combinatorics (symbolic method)	The payoff chapter: read a combinatorial structure's <i>definition</i> straight off into a generating function. Hardest, most powerful.

Part III — Applications to real structures (Ch 6-9)

Module	Chapter	Topic	Data-structures payoff
6	6	Trees	★ Binary search trees, Catalan numbers, path length — directly the DS you use.
7	7	Permutations	★ Sorting analysis: inversions, insertion & selection sort, cycles.
8	8	Strings & Tries	★ String search, Knuth-Morris-Pratt, tries.
9	9	Words & Mappings	★ Hashing analysis, birthday paradox, coupon collector, random maps.

On-demand expansions (opened when needed)

- Dense chapters (2, 3, 4, 5) may split into `-a / -b` parts when a single module gets too big.
- Any point where you want more depth — just ask, and we open a focused sub-chapter there.

Companion resources (free)

- **Coursera — "Analysis of Algorithms" (Robert Sedgewick, Princeton).** Same author, same book, lecture videos. Lighter than the book; great first pass per chapter.
- **Booksite:** `aofa.cs.princeton.edu` . Full lecture slides, extra exercises, and errata. Our problem sets draw on its exercises.
- **The book PDF** (this folder) — the source of truth; we read it section by section.

Suggested rhythm

No clock, but a sane loop per module:

1. Skim the chapter intro + watch the matching Coursera lecture (context).
2. Read my notes; work the derivations by hand, don't just read them.
3. Do the problem set; check against the worked solutions.
4. Mark it in `PROGRESS.md` ; open an expansion if anything still feels shaky before moving on.

Chapters 1-4 are the investment; from Chapter 5 on, the tools start paying dividends fast.

Pace & sessions

This is a **side course, done in limited free time**, so pace is deliberate and that's fine. We work in **30-45 minute sessions**, each a single bite, so progress is easy to see and resume.

English is not the reader's first language (prior study was in Hebrew), and *improving mathematical/professional English is a deliberate secondary goal of this course*. The notes are therefore written in a **high, precise academic register** — proper technical vocabulary, not a simplified version — with brief inline glosses only for genuinely advanced terms.

What one session is — pick one:

- read ~one subsection of the notes and redo its derivation by hand, **or**
- work 2-4 exercises and check them, **or**
- one focused derivation walk-through (e.g. the quicksort solve).

Consistency beats length: a real 30-minute session every couple of days outruns rare marathons.

Estimated sessions per module (a careful, unhurried pace, with re-reading built in):

Module	Topic	~Sessions
0	Foundations	3
1	Analysis of Algorithms	4
2	Recurrence Relations	8
3	Generating Functions	10
4	Asymptotic Approximations	10
5	Analytic Combinatorics	7
6	Trees	11
7	Permutations	8
8	Strings & Tries	8
9	Words & Mappings	8
—	Total	≈ 77 (budget 75-90 with buffer)

At ~40 min each that's roughly **50-60 hours** of focused study — a genuine upper-level course.

Time to finish, by cadence:

Sessions / week	Finish in
1	~1.5 years
2	~9 months
3	~6 months

A sustainable target given the constraints is **2 sessions/week $\approx$ 9 months** — but the only number that matters is the streak, not the deadline. We reassess every few sessions using the session log in `PROGRESS.md`.